

Free Website Audit Toolkit

A \$0 stack for scanning 20+ websites for technical mistakes, performance issues, security gaps, and infrastructure details.

Core Automation Stack

Tool	Cost	Purpose	Scale Rating
Lighthouse CLI	\$0	Performance, accessibility, SEO, best practices	★★★★★
Playwright	\$0	Browser automation and functional testing	★★★★★
Python	\$0	Automation and orchestration of the whole pipeline	★★★★★
Requests / httpx	\$0	HTTP crawling and status/header checks	★★★★★
BeautifulSoup / lxml	\$0	HTML parsing and structural analysis	★★★★★
OWASP ZAP	\$0	Automated web security testing	★★★★★
SecurityHeaders.com	\$0	HTTP security header checking	★★
Mozilla Observatory	\$0	Security configuration scoring (API available)	★★★
Wappalyzer (free)	\$0	Basic technology / CMS detection	★★★
WhatRuns (free)	\$0	Alternative tech detection — scripts, widgets, trackers	★★★
DNS / WHOIS utilities	\$0	Nameservers, hosting, registrar, certificates, subdomains	★★★★
SQLite / PostgreSQL	\$0	Store and query results across all scanned sites	★★★★★

Why These, Not Overlapping Duplicates

Wappalyzer — fastest for a quick, single-site technology identification (browser-based). **WhatRuns** — free alternative, better at catching scripts, widgets and third-party trackers Wappalyzer sometimes misses. Use one or both, not a paid tool, for basic tech ID across 20 sites.

SecurityHeaders.com and **Mozilla Observatory** both grade HTTP security configuration (HSTS, CSP, X-Frame-Options, etc.) — Observatory has a scriptable API, making it the better fit for checking 20 sites in one automated pass; SecurityHeaders is fine for occasional single-site spot checks.

OWASP ZAP goes further than header checks — it actively scans for real vulnerabilities (XSS, injection points, misconfigurations), and can be run headlessly against a URL list.

Recommended Pipeline for 20 Sites

1. Maintain the 20 URLs in a simple list or table (SQLite/PostgreSQL, or even a CSV to start).
2. Loop through the list in Python, calling Lighthouse CLI on each URL — capture performance, accessibility, SEO and best-practice scores plus flagged issues.
3. In the same loop, use Requests/httpx + BeautifulSoup to pull page HTML, check status codes, broken links, and meta tags.

4. Run each URL through Mozilla Observatory's API (or SecurityHeaders) to score security headers.
5. Run DNS/WHOIS checks per domain — hosting provider, nameservers, SSL certificate validity, subdomains.
6. Optionally run OWASP ZAP's headless baseline scan for deeper security issues.
7. Use Wappalyzer/WhatRuns lookups to tag each site's tech stack, so problems can be linked back to the platform causing them (e.g. outdated WordPress plugin).
8. Store every result in SQLite/PostgreSQL, keyed by URL, so you get one queryable table of every mistake across all 20 sites.

All tools listed above are free for this scale of use (20 sites). Everything here can be orchestrated as a single Python script or wired into an n8n workflow, matching your existing automation setup.